

Write the code used to perform the following tasks from the MongoDB shell:

1. Create a database called zoo.

```
test> use zoo
```

2. Insert a collection called animals.

```
zoo> db.createCollection("animals")
```

3. Confirm that the database and the collection have been successfully created.

```
zoo> show dbs  
zoo> show collections
```

4. Insert the documents contained in the animals.json file.

```
zoo> db.animals.insertMany([  
  
  {  
    "_id": 1,  
    ...  
    {  
      "_id": 70,  
      "name": "Horn",  
      "age": 7,  
      "gender": "Male",  
      "species": "Rhinoceros",  
      "habitat": "Grasslands",  
      "birthplace": "Kenya",  
      "parents": [  
        { "name": "Unknown", "gender": "Male" },  
        { "name": "Unknown", "gender": "Female" }  
      ],  
      "caretakers": [{ "idCaretaker": 614, "nameCaretaker": "Nathan Hall" }]  
    }  
  ])  
  {  
    acknowledged: true,  
    ...  
  }  
])
```

5- Write a MongoDB query to find animals that have Rainforest as their habitat and are attended by a caretaker named Emily Brown.

```
zoo> db.animals.find({  
  $and: [  
    { habitat: "Rainforest" },  
    { "caretakers.nameCaretaker": "Emily Brown" }  
  ]  
})
```

6- Write a MongoDB query to display only the name, species, and habitat of all animals sorted by their species in ascending order.

```
zoo> db.animals.find(
  {},
  { _id: 0, name: 1, species: 1, habitat: 1 }
).sort({ species: 1 })
```

7 .Write a MongoDB query to find animals with the species Tiger and display only their name and age, sorted by age in descending order.

```
zoo> db.animals.find(
  { species: "Tiger" },
  { _id: 0, name: 1, age: 1 }
).sort({ age: -1 })
```

8. Write a MongoDB query to insert the next animal into the animals collection:

name: "Leo",
age: 7,
gender: "Male",
species: "Lion",
habitat: "Savanna",
birthplace: "Masai Mara, Kenya",
parents: "Unknown", gender: "Male", "Unknown", gender: "Female"
caretakers: id: 501, name: "Jake Adams"]]

```
zoo> db.animals.insertOne({
  name: "Leo",
  age: 7,
  gender: "Male",
  species: "Lion",
  habitat: "Savanna",
  birthplace: "Masai Mara, Kenya",
  parents: [
    { name: "Unknown", gender: "Male" },
    { name: "Unknown", gender: "Female" }
  ],
  caretakers: [
    { idCaretaker: 501, nameCaretaker: "Jake Adams" }
  ]
})
{
  acknowledged: true,
  insertedId: ObjectId('69f0cae1b44f30ce3c3682d1')
}
```

9. Write a MongoDB query to update the age (to 8) of the animal you just inserted using

the \$set operator

```
zoo> db.animals.updateOne (
  { name: "Leo" },
  {$set: { age: 8 } }
)
```

10. Write a MongoDB query to rename the habitat field to environment for all animals in the animals collection.

```
zoo> db.animals.updateMany (
  {},
  {$rename: {habitat: "environment"} }
)
```

11- Write a MongoDB query using the aggregation framework to find animals aged less than 10 and sort them alphabetically by their name.

```
zoo> db.animals.aggregate ( [
  {$match: { age: {$lt: 10} } },
  { $sort: { name: 1} }
] )
```

12. Write a MongoDB query using the aggregation framework to group animals by their species and count the number of animals in each species.

```
zoo> db.animals.aggregate ( [
  {$group: { _id: "$species",
  count: { $sum: 1 } } }
] )
```

13. Write a MongoDB query to create a new collection called Caretakers by extracting all caretakers from the animals collection. Ensure that caretakers do not repeat and include an array of the animals they attend to.

```
zoo> db.animals.aggregate([
  { $unwind: "$caretakers" },
  { $group: {
    _id: "$caretakers.idCaretaker",
    nameCaretaker: { $first: "$caretakers.nameCaretaker" },
    animals: { $push: "$name" }
  }
},
  { $project: {
    _id: 0,
    idCaretaker: "$_id",
    nameCaretaker: 1,
    animals: 1
  }
},
  { $out: "Caretakers" }
])
```

14. Write a MongoDB query to find the top 3 oldest animals.

```
zoo> db.animals.find()  
      .sort  
        ({ age: -1 })  
      .limit(3)
```

15. Write a MongoDB query to display animals between the 6th and 12th entries in the collection based on their insertion order.

```
zoo> db.animals.find()  
      .skip(5)  
      .limit(7)
```

16. Write a MongoDB query using the aggregation framework to create a new collection called Habitats. This collection should group animals by their environment (formerly habitat) and include the following fields.

- The environment name.
- The total number of animals in that environment.
- An array of unique species in that environment, sorted alphabetically.
- An array of unique caretakers who attend animals in that environment.

José Manuel Rebollo Bernal

Task 07 DB

28/04/2026

```
zoo> db.Habitats.find()
[
  {
    _id: ObjectId('679370346cee80a6809e83a7'),
    totalAnimals: 11,
    environment: 'Grasslands',
    species: [ 'Buffalo', 'Elephant', 'Kangaroo', 'Rhinoceros' ],
    caretakers: [
      'Alice Brown',
      'Alice Green',
      'Alice White',
      'Emily Brown',
      'Emily Green',
      'Jake Adams',
      'John Smith',
      'Mark Taylor',
      'Nathan Hall'
    ]
  },
  {
    _id: ObjectId('679370346cee80a6809e83a8'),
    totalAnimals: 10,
    environment: 'Savanna',
    species: [ 'Giraffe', 'Lion' ],
    caretakers: [
      'Alice Green',
      'Emily Brown',
      'Jake Adams',
      'John Smith',
      'Mark Taylor',
      'Michael Davis',
      'Sophia Green'
    ]
  },
  {
    _id: ObjectId('679370346cee80a6809e83a9'),
    totalAnimals: 3,
    environment: 'River',
    species: [ 'Hippopotamus' ],
    caretakers: [ 'Alice Green', 'Mark Brown', 'Sophia Taylor' ]
  },
]
```

```
zoo> db.animals.aggregate([
  { $unwind: "$caretakers" },
  { $group: {
    _id: "$environment",
    totalAnimals: { $sum: 1 },
    species: { $addToSet: "$species" },
    caretakers: { $addToSet: "$caretakers.nameCaretaker" }
  } },
  { $project: {
    _id: 0,
    environment: "$_id",
    totalAnimals: 1,
    species: 1,
    caretakers: 1
  } },
  { $out: "Habitats" }
])
```